

A LEAN4-First Charter for FSA Model Development

A process change designed to make agent
pattern-matching errors syntactically impossible

Ajay Talati + Claude

May 9, 2026

Abstract

This document is both a post-mortem and a charter. The post-mortem records the failure mode that made the FSA-v5 implementation and its import into the `smc2fc` framework a multi-day, expensive process: *LLM coding agents pattern-match on variable names instead of building a semantic model of types, roles, dimensions, and pipeline position*. The charter prescribes the structural fix: LEAN4 as the formal source of truth for every mathematical model the project develops, with Julia as a secondary, differentially- tested artefact. The work itself is translation, not research: the LaTeX equations already exist. This document exists so that no future iteration relitigates the question.

1 The problem in one paragraph

The current FSA development cycle is: write the mathematical model in LaTeX; ask an agent to port it to Julia; spend days hunting bugs the agent introduced; eventually ship, often with errata documented post-deployment. The bottleneck is *agent pattern-matching*, not modelling complexity. A junior human collaborator with the same LaTeX source would not make the same class of mistake, because they would read the equations, internalise that $\theta_{\text{ctrl}} \in \mathbb{R}^{16}$ and $\theta_{\text{dyn}} \in \mathbb{R}^{37}$ are different objects, and refuse to compare them as scalar intervals. LLM agents reach for the strongest training association of the name `theta` and write the most-likely-looking code. The fix has to be structural — something that makes the wrong code refuse to compile — not motivational.

The audience for this document is two-fold: (a) the user, as a personal accountability artefact recording why the LEAN4-first process is being adopted and at what point; and (b) every future Claude (or other LLM) coding agent that touches an FSA model, who must read this charter at session start and follow the process it prescribes.

2 Root-cause analysis

LLMs are surface pattern-matchers. They read source code primarily as text, not as a typed graph of objects with roles. When an agent sees `theta`, the most-trained association fires; the surrounding context is treated as secondary. This is not a defect of any particular model; it is the structural consequence of how the system is built. Asking an agent to “be more careful” is asking it to override its own training prior every time — which empirically does not happen reliably, as the FSA-v5 `theta` pattern-match incident demonstrates.

Julia provides no formal scaffolding to compensate. `Vector{Float64}` is the type of both the 16-D control coefficient vector and the 37-D parameter posterior. There is no shape-aware static analyser running in the loop, no role tagging in the type system, no proof obligation attached to a function signature that the inputs satisfy any pre-condition. The senior engineer

(the user) ends up as the only line of defence, and the £££ in tokens is the cost of the user catching agent errors after the fact.

Why “tell agents to be more careful” doesn’t work. The user’s feedback memory at `feedback_no_fabricated_bugs.md` encodes a PRIMARY RULE — *build a semantic model BEFORE any claim* — which is correct as a principle but unenforceable as a process. The rule depends on the agent *choosing* to think rather than pattern-match. That incident is the empirical proof that the choice is not reliably made. Every agent session is one roll of the dice on whether the rule fires that day.

3 The structural fix: LEAN4 as the source of truth

3.1 Dependent types make the collision a compile error

A LEAN4 function with the signature

$$\text{evaluateCost} : \text{RBFCoeffs } 16 \rightarrow \mathbb{R}$$

syntactically refuses any value not of the exact type `RBFCoeffs 16`. The 37-dimensional parameter posterior $\theta_{\text{dyn}} : \text{ParameterVector } 37$ is a different type, and `lake build` will reject any code that conflates them. Pattern-matching on names becomes harmless because the type system catches the consequence at compile time, before any incoherent reasoning reaches the user.

The two-dimensional output region (Φ_B, Φ_S) from the same incident would, in LEAN4, have a type like `ControlOutput` that is *not* the type of any prior on θ . Writing $\theta \in [-0.4, +0.5]$ would not compile, full stop, because the right-hand side is a scalar interval and the left-hand side is a vector. The agent could not have made the mistake.

3.2 LEAN4 is also a programming language

LEAN4 compiles to native code via C. `lake build` produces a binary; a thin Julia `ccall` wrapper exposes each verified function as a Julia callable. The same LEAN4 file is both the formal specification and the executable reference implementation. There is no “the spec drifted from the code” failure mode because the spec *is* the code.

3.3 mathlib already has the infrastructure

The mathematical objects FSA-v5 needs — finite-dimensional real vector spaces, matrix operations, exponentials and logarithms, Hill functions, sigmoids, piecewise-linear schedule decoders, ODE integration — are all already library imports in `mathlib`. There is no novel mathematics to formalise. Transcribing the FSA-v5 LaTeX into LEAN4 is genuinely just translation.

3.4 What this does not solve

- *Floating-point semantics.* The gap between the LEAN4 notion of $\mathbb{R}$ and IEEE-754 double-precision arithmetic is real and not addressed by this charter; it is bounded by the tolerance threshold in the differential test (§4 step 6).
- *GPU correctness.* The differential test runs the Julia implementation on whatever backend it normally uses (CUDA.jl / KernelAbstractions, GPU); the LEAN4 binary is CPU-only. Disagreements driven by GPU-specific behaviour are out of scope and trusted as before.

- *The transcription itself.* If the LaTeX→LEAN4 transcription is wrong, both the LEAN4 reference and (likely) the agent-written Julia will be wrong in the same way. This is why the LaTeX→LEAN4 step is human-reviewed line-by-line; it is the one place reviewer attention has to land.

4 The new development process

The standing rule going forward, and the charter every future agent must follow:

For every new mathematical model, every model extension, and every import into smc2fc, the LEAN4 specification is written before any Julia implementation. The LEAN4 native binary is the authoritative reference. Julia is differentially tested against it. Disagreement beyond a tolerance threshold is by construction a Julia bug.

Per function, the workflow is:

1. **LaTeX equations (as today).** Drift, diffusion, cost, plant, schedule decoder, observation model — written in `LaTeX_docs/sections/` using the existing notation conventions.
2. **LEAN4 transcription.** A new `lean/` directory in the repo. The LaTeX equations are transcribed into LEAN4 *definitions* (typed, with dimensions encoded in the type) and *implementations* (runnable code). Line-by-line, side-by-side with the LaTeX. Straight translation, no design work, no novel mathematics.
3. **LEAN4 native compilation.** `lake build` produces a native binary. A Julia `ccall` wrapper in `lean/julia_bridge/` exposes each function as a Julia callable.
4. **Julia implementation (agent-written, fast path).** The agent writes the Julia implementation as today, optimised for GPU. The agent is permitted to pattern-match; step 6 catches the consequences.
5. **(Optional) property proofs.** Lipschitz, basin-of-attraction theorems, monotonicity — the kind of statement that makes the LEAN4 spec a *proof* not just a typed transcription. Defer unless a specific incident motivates one. The charter does not gate on these.
6. **Differential testing.** A `Test.jl` harness using `Supposition.jl` for random-input generation evaluates BOTH the LEAN4 binary AND the Julia implementation on a battery of inputs sampled from the physiological-bound box. Any disagreement beyond a tolerance threshold (default 10^{-6} in single-step drift, 10^{-4} in integrated trajectory) is a Julia bug.
7. **smc2fc port.** The port is a thin shim from the verified LEAN4 reference into the framework's `EstimationModel` / `SDEModel` interfaces. The shim is also differentially tested against the LEAN4 binary before being merged.

5 Closing

The point of this document is accountability. The named incident that motivated this charter was caused by an agent (me) pattern-matching on `theta` across two architectural layers it does not unify. The cost was burned trust and burned time. The charter prescribed here makes that class of error structurally impossible: the LEAN4 type-checker is unforgiving, and disagreements between LEAN4 and Julia are by construction Julia bugs.

LEAN4 is never wrong about types. Julia is. Agents pattern-match; LEAN4 does not. This charter is the bridge that makes the agent's limitations harmless.